BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

Nesne Yönetimi, Güvenlik ve Sihirli Metotlar

EBRU AYDOĞAN

Encapsulation (Kapsülleme) ve Properties

- **Problem:** Bir Product sınıfınız olduğunu düşünün. Bir kullanıcının ürün fiyatını -1000 TL yapması veya bir banka bakiyesini 0'dan küçük yapması sistemin çökmesine veya hatalı veri oluşmasına neden olur.
- **Encapsulation:** Nesnenin içindeki veriyi dış dünyaya karşı gizlemek ve veriye sadece "onaylanmış kanallardan" erişilmesini sağlamaktır.
- **Analoji:** Bir evin elektrik panosu. Pano kilitli bir kutudadır (kapsülleme). İçindeki kablolara (verilere) doğrudan elinizi sokamazsınız. Sadece sigortayı açıp kapatmak (metot/property) için izin verilmiştir.

Python'da Erişim Kontrolü (_ ve __)

- Python, diğer diller (Java/C#) gibi katı private anahtar kelimelerine sahip değildir; bunun yerine **görgü kuralları (convention)** vardır.
  - **Public (name):** Herkes erişebilir, herkes değiştirebilir.
  - **Protected (_name):** "Lütfen dışarıdan erişme, sadece sınıf ve alt sınıflar için" mesajı verir. (Tek alt çizgi).
  - **Private (__name):** "Buna kesinlikle dışarıdan erişme!" mesajı verir. (Çift alt çizgi). Python burada *Name Mangling* yaparak ismi değiştirir, erişimi zorlaştırır.

@property ile Kontrollü Erişim (Getter & Setter)

- Eskiden (başka dillerde) get_price() ve set_price() metotları yazardık. Python'da ise **@property** decorator'ını kullanarak bir metodu "sanki bir değişkenmiş gibi" çağırabiliriz.
  - **Getter:** Veriyi okumamızı sağlar (veya okurken modifiye etmemizi).
  - **Setter:** Veriyi atarken **kontrol etmemizi** (validation) sağlar.

Kod Uygulaması – Güvenli Fiyat Yönetimi

- Doğrudan müdahale edildiğinde hatalı veri girilebilen bir yapıyı, güvenli yapıya dönüştürelim:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        # Fiyatı doğrudan atamak yerine, setter metodunu çağırıyoruz
        self.price = price
```

BTK AKADEMİ

Kod Uygulaması – Güvenli Fiyat Yönetimi

```python
# Getter: Fiyatı okurken kullanılır
@property
def price(self):
    return self.__price

# Setter: Fiyatı atarken kullanılır (Kontrol noktası!)
@price.setter
def price(self, value):
    if value < 0:
        print("Hata: Fiyat negatif olamaz!")
        self.__price = 0 # Varsayılan değer
    else:
        self.__price = value
```

BTK AKADEMİ

Kod Uygulaması – Güvenli Fiyat Yönetimi

```
# Kullanım:
p1 = Product("Laptop", 20000)
print(p1.price) # 20000

# Hatalı atama yapmayı deneyelim:
p1.price = -500 # "Hata: Fiyat negatif olamaz!" mesajı gelir.
print(p1.price) # 0
```

BTK AKADEMİ

Neden @property Kullanmalıyız?

- **Veri Doğrulama (Validation):** Yanlış veri girişini (negatif fiyat, geçersiz e-posta vb.) engelleriz.
- **Backwards Compatibility:** İleride price mantığını değiştirmek istersek (örneğin fiyatı veritabanından çekmek gibi), kullanıcıların p1.price kodunu değiştirmesine gerek kalmaz.
- **Temiz Kod:** p1.set_price(-500) gibi karmaşık metot çağrıları yerine p1.price = -500 gibi doğal bir atama söz dizimi kullanırız.

Special (Magic/Dunder) Methods

- **Dunder Ne Demek?:** "Double Underscore" kelimelerinin kısaltmasıdır. Metot isimleri başında ve sonunda iki alt çizgi (__) ile başlar ve biter.
- **Neden Sihirli?:** Çünkü biz bu metotları objem.metot() şeklinde çağırmayız. Python, kendi içindeki yerleşik fonksiyonları (örneğin print(), len()) kullandığımızda bu metotları **arka planda otomatik olarak** tetikler.
- **Analoji:** Python bir "kulüp" gibidir. Kendi yazdığınız sınıflar bu kulübe ilk başta yabancıdır. __str__, __len__ gibi "sihirli" metotları yazdığınızda, Python sınıfınızı kulübe kabul eder ve diğer tüm yerleşik (built-in) fonksiyonlarla uyumlu çalışmasını sağlar.

__str__ ile Nesneyi Tanımlamak

- **Problem:** Bir sınıf oluşturup nesneyi print() ile yazdırdığımızda karşımıza çıkan "<__main__.Movie object at 0x00...>" gibi karmaşık hafıza adreslerinden nasıl kurtuluruz?
- **Çözüm:** __str__ metodunu sınıfa ekleyerek nesnenin "insan tarafından okunabilir" halini tanımlarız.

BTK AKADEMİ

__str__ ile Nesneyi Tanımlamak

```
class Movie:
    def __init__(self, title, director, duration):
        self.title = title
        self.director = director
        self.duration = duration

    # Print() çağrıldığında bu metot çalışır
    def __str__(self):
        return f"{self.title} filmi, {self.director} tarafından yönetilmiştir."

m = Movie("Inception", "Christopher Nolan", 148)
print(m) # Çıktı: Inception filmi, Christopher Nolan tarafından yönetilmiştir.
```

BTK AKADEMİ

__len__ ile Nesne Uzunluğu

- Problem: Listelerin len() uzunluğu vardır. Peki ya kendi film sınıfımızın da bir "uzunluğu" olmasını istersek? (Örn: Filmin süresi).
- Çözüm: __len__ metodunu kullanarak len(film_nesnesi) çağrısını özelleştiririz.

__len__ ile Nesne Uzunluğu

```
    # len() çağrıldığında bu metot çalışır
    def __len__(self):
        return self.duration

m = Movie("Inception", "Christopher Nolan", 148)
print(len(m)) # Çıktı: 148
```

BTK AKADEMİ

__del__ ve Nesnenin Yaşam Döngüsü

- **Kavram:** Nesneler bellekte yer kaplar. Bir nesne silindiğinde (örneğin program kapandığında veya del anahtar kelimesi kullanıldığında) arka planda bir "temizlik" yapmak isteyebiliriz.
- **Uygulama:** Nesne bellekten silinmeden hemen önce çalışacak olan metottur.

__del__ ve Nesnenin Yaşam Döngüsü

```
    def __del__(self):
        print("Film objesi silindi.")

m = Movie("Inception", "Christopher Nolan", 148)
del m # Çıktı: Film objesi silindi.
```

BTK AKADEMİ

__eq__ ile Eşitlik Mantığını Tanımlamak

- **Problem:** Python'da varsayılan olarak iki nesneyi == ile karşılaştırırsanız, Python nesnelerin içeriğine bakmaz; **bellek adreslerine (id)** bakar. Eğer iki farklı nesne oluşturduysanız, değerleri aynı olsa bile Python False (eşit değiller) der.
- **Çözüm:** __eq__ (equals) metodunu kullanarak Python'a "Ne zaman eşit sayılmalıyız?" kuralını biz tanımlarız.
- **Analoji:** İki tane "Inception" filmi nesnesi düşünün. İkisi de farklı kutularda (bellekte) duruyor ama ikisinin de başlığı "Inception". Bizim için bunlar "aynı" filmdir. __eq__ bu mantığı kurmamızı sağlar.

__eq__ ile Eşitlik Mantığını Tanımlamak

```
    # Eşitlik kuralı: "İsimleri aynıysa, bu nesneler eşittir"
    def __eq__(self, other):
        # 'other', karşılaştırdığımız diğer nesnedir (== sağındaki)
        return self.title == other.title

# Kullanım:
m1 = Movie("The Matrix", "Wachowski", 136)
m2 = Movie("The Matrix", "Wachowski", 136)

print(m1 == m2)  # Artık True döner!
```

BTK AKADEMİ

Uygulama 1: E-Ticaret Sepeti Tasarımı

- **Senaryo:** Bir e-ticaret sitesi için backend geliştiren bir ekipte yer alıyorsunuz. Sizden, ürünleri yönetebileceğiniz ve bu ürünleri bir sepete ekleyip toplam tutarı hesaplayabileceğiniz nesne yönelimli (OOP) bir sistem tasarlamanız isteniyor.

BTK AKADEMİ

Uygulama 1: E-Ticaret Sepeti Tasarımı

- Gereksinimler (Lütfen hepsini uygulayın):

1. Product (Ürün) Sınıfı:
   - Her ürünün ad ve fiyat bilgisi olmalıdır.
   - **Kural:** Fiyat bilgisi asla negatif olamaz. (Bunu property ve setter ile kontrol edin).
   - **Kural:** İki ürünün adı aynıysa, bu ürünler "aynıdır" kabul edilmelidir (Bunu __eq__ ile sağlayın).

BTK AKADEMİ

Uygulama 1: E-Ticaret Sepeti Tasarımı

2. ShoppingCart (Alışveriş Sepeti) Sınıfı:

- Sepet, içerisine Product nesneleri ekleyebilmelidir.
- **Mantık:** Sepete aynı isimli bir ürün eklenirse; yeni bir ürün eklemek yerine, ürünün adet bilgisini artırın veya "bu ürün zaten sepette" uyarısı verin (Tercih sizin).
- **Toplam Tutar:** Sepetteki tüm ürünlerin fiyatları toplamını veren total_price adında bir @property yazın.
- **Gösterim:** Sepeti print() ile yazdırdığınızda, içindeki tüm ürünleri ve adetlerini şık bir liste halinde göstermelidir (__str__).
- **Uzunluk:** Sepetteki toplam ürün çeşidi (benzersiz ürün sayısı) len(sepet) denildiğinde dönmelidir (__len__).

BTK AKADEMİ